

Swift Pay QA Review

Scan to Pay QR Code Feature, Quality Assurance Review Report

Tester: Jado Fils

Date: 2026-08-15

Feature under review: Scan to Pay (QR code payments), Swift Pay mobile app

References: Feature specification and acceptance criteria (AmaliTech Software Testing and Quality Assurance training repository); interactive prototype, Figma file "Payswift-Ambiguity-flow (QAs)".

Table of Contents

Table of Contents	2
Acceptance Criteria (AC)	3
Section 1: Positive Test Case	5
Section 2: Negative / Edge Test Case	7
Section 3: Bug Report	10
BUG-001	10
BUG-002	11
BUG-003	13
BUG-004	15
Section 4: Process Improvement Suggestion (Shift-Left Testing)	17
What could have been done earlier	17
How it would prevent the issue	17
Relation to Shift-Left Testing	18

Acceptance Criteria (AC)

ID	User Action	System Response
AC-01	User taps the "Scan" icon from the app home screen	The app opens the phone camera in scan mode
AC-02	User points camera at a valid merchant QR code	System decodes QR code and navigates to Payment Confirmation screen
AC-03	On the Payment Confirmation screen	Merchant name (e.g., "Kofi's Cafe") is displayed at the top
AC-04	User enters amount to pay	User can input amount and "Confirm Payment" button becomes active
AC-05	User taps "Confirm Payment"	User is prompted to enter 4-digit transaction PIN
AC-06	User enters correct PIN	Transaction is processed quickly
AC-07	Upon successful transaction	Success message appears showing amount and merchant name
AC-08	Upon failed transaction (e.g., incorrect PIN)	Error message displayed and user returned to

		confirmation screen
AC-09	QR code contains embedded amount	User cannot edit the amount

QA Review Submission Requirement: this review is submitted using the provided Swift Pay QA Review Template.pdf, following the suggested four-section structure below.

Section 1: Positive Test Case

Test Case ID	TC-POS-01
Title	Successful QR scan and payment completion
Preconditions	1. User has the Swift Pay app installed and is logged in. 2. User has a linked, sufficiently funded payment method. 3. Camera permission is granted to the app. 4. A valid, active merchant QR/product is available for scanning (e.g., "Kofi's Cafe").
Steps	1. From the app home screen, browse the product/merchant list and select one. 2. Tap into the selected item to open the scanner; the camera opens and scans the merchant's QR code. 3. On the Payment Confirmation screen, verify the merchant name is displayed at the top. 4. Enter a valid payment amount within the available balance. 5. Tap Confirm Payment.

	6. Enter the correct 4-digit transaction PIN. 7. Observe the outcome.
Expected Result	The camera opens and decodes the QR code (AC-01, AC-02); the Payment Confirmation screen shows the merchant name (AC-03); the Confirm Payment button gates on a valid amount (AC-04); PIN entry is prompted (AC-05); and, on a correct PIN, a success message appears showing the amount and merchant name, with the user routed to a receipt or home screen (AC-06, AC-07).
Actual Result (observed, manual walkthrough)	Steps 1 and 2 work as expected: selecting a product opens the camera and it scans successfully. At step 4, the amount field accepts entry with no validation at all (see BUG-002). After entering the PIN at step 6, no success or confirmation message appears and there is no next screen; the flow simply dead-ends (see BUG-001).
Verdict	FAIL. Blocked by BUG-001; AC-07 cannot be verified because the

	prototype never reaches a success state.
--	--

Section 2: Negative / Edge Test Case

Test Case ID	TC-NEG-01
Scenario	User enters a payment amount greater than the available account balance (and separately, a zero, negative, or non-numeric amount); the app should reject the amount with a clear error message and guidance, but instead accepts it silently and lets the user proceed.
Preconditions	1. User has scanned a valid merchant QR code and reached the Payment Confirmation screen (per AC-02, AC-03).
Steps	1. On the Payment Confirmation screen, enter an amount noticeably higher than the linked account's available balance. 2. Observe whether the Confirm Payment button and/or an inline message prevents proceeding.

	3. Repeat with 0, a negative number, and non-numeric characters. 4. If not blocked at any point, tap Confirm Payment and continue to PIN entry as in TC-POS-01.
Expected Result	The amount field should reject non-numeric input outright, and the Confirm Payment button should stay disabled, with a clear inline error message and guidance (for example, "Amount exceeds available balance"), for an over-balance amount, 0, or a negative amount (AC-04 implies a valid amount is required to activate the button, which presupposes validation).
Actual Result (observed)	No validation and no error message or guidance of any kind is shown. The reviewer entered an amount greater than the available balance and was able to proceed straight to PIN entry with no warning at all; the same held for zero, negative, and non-numeric entries (see BUG-002).
Verdict	FAIL.

Note on scope: the assignment's example negative scenarios (invalid QR code, network failure, incorrect PIN) could not be exercised as cleanly as this one, because the prototype's happy path itself dead-ends before reaching any success or failure feedback state (BUG-001), so failure-path messaging (AC-08) cannot be verified live either. Those remain spec-level gaps; see BUG-004. As this is a Figma prototype rather than a working backend, some of this may reflect prototyping limits rather than a deliberate design choice, but the mockup should still visually communicate an error state for QA sign-off to be meaningful; see BUG-002.

Section 3: Bug Report

BUG-001

Title	No payment confirmation message and no next step after PIN entry
Severity	Critical. This is the single most important moment in a payments flow (did the money move or not), and the prototype gives the user no feedback at all, directly violating AC-07.
Priority	High. Must be fixed or wired in the mockup before any development sign-off, since developers would have nothing to build against for this state.
Environment	Figma prototype "Payswift-Ambiguity-flow (QAs)", Present mode, manual walkthrough by reviewer.
Steps to Reproduce	1. Open the prototype in Present mode. 2. From the home screen, select a product/merchant and tap it to trigger the camera scan. 3. On the Payment Confirmation screen, enter any amount and tap Confirm Payment.

	4. Enter a 4-digit PIN. 5. Observe the screen.
Expected Result	Per AC-07, a success message should appear showing the transaction amount and merchant name, and per AC-08 an incorrect PIN should show an error and return the user to the confirmation screen. Either way, something should happen.
Actual Result	Nothing happens. No success message, no error message, and no navigation to any further screen. The flow terminates silently at PIN entry, so neither AC-07 nor AC-08 can be verified against the mockup.

BUG-002

Title	Payment amount field has no input validation and gives no error message or guidance
Severity	High. An unvalidated amount field on a money-movement screen is a direct correctness and security concern; amounts exceeding the available

	balance, zero, negative, and non-numeric input should never reach a payment processor, and a user who overpays by mistake deserves a clear message telling them why they were stopped.
Priority	High
Environment	Figma prototype "Payswift-Ambiguity-flow (QAs)", Present mode, manual walkthrough by reviewer.
Steps to Reproduce	1. Reach the Payment Confirmation screen as in BUG-001, steps 1 and 2. 2. Enter an amount greater than the account's available balance. 3. Separately, try 0, a negative number, and non-numeric text. 4. Observe that Confirm Payment activates and the flow proceeds regardless of the value entered, with no error message or guidance shown.
Expected Result	AC-04 implies the Confirm Payment button activates on a valid amount, implying invalid values (over-balance, zero, negative, non-numeric) should be rejected with a clear, specific inline

	error message and guidance, and should keep the button disabled.
Actual Result	The field accepts entry with no validation of any kind and displays no error message or guidance; the reviewer proceeded to PIN entry without a legitimate amount being enforced, including with an amount above the available balance.

BUG-003

Title	Contradictory acceptance criteria for payment amount entry (AC-04 vs. AC-09)
Severity	High. The ambiguity is in a core transaction path; whichever behavior is built, the other half of the spec is simply wrong.
Priority	High. Should be clarified before development starts on this screen.
Environment	Specification review (Software Testing and Quality Assurance training repository).
Steps to Reproduce	1. Read AC-04: "User enters amount to

	pay; user can input amount and 'Confirm Payment' button becomes active." 2. Read AC-09: "QR code contains embedded amount; user cannot edit the amount." 3. Compare against the Payment Confirmation screen, which in the observed prototype is a freely editable amount field, consistent with AC-04 but contradicting AC-09.
Expected Result	The specification should describe one unambiguous amount-entry behavior. If Swift Pay supports both static QR codes (merchant-preset amount) and dynamic QR codes (amount entered manually), the spec should say so explicitly and define how the confirmation screen differs between the two, for example, editable and required for dynamic codes, read-only or pre-filled for static codes.
Actual Result	AC-04 describes an editable amount field that gates the Confirm Payment button, implying the amount is always

	user-entered. AC-09 describes an embedded, non-editable amount, implying the amount is never user-entered. The prototype itself resolves this in favor of AC-04 (the amount field observed is editable), meaning AC-09 as written does not match either the spec's own intent or the mockup.
--	---

BUG-004

Title	No acceptance criteria for invalid/unreadable QR codes, network failure, or repeated PIN failure lockout
Severity	Medium. Common real-world failure conditions for a payments feature; their absence risks an inconsistent or insecure implementation, but does not block the happy path.
Priority	Medium
Environment	Specification review (AC-01 through AC-09).
Steps to Reproduce	1. Review AC-01 through AC-09 for any criterion covering: (a) an invalid, expired, or unrecognized QR code; (b)

	loss of network connectivity after Confirm Payment is tapped; (c) behavior after three or more consecutive incorrect PIN attempts. 2. Confirm none of AC-01 through AC-09 addresses any of these three cases.
Expected Result	The spec should define an explicit error state and message for each case above, distinct from the generic "incorrect PIN" error in AC-08, for example "QR code not recognized," "Connection lost, transaction not completed," and a PIN-attempt lockout or cooldown rule.
Actual Result	None of these cases appear anywhere in AC-01 through AC-09, leaving the failure-mode behavior of a money-movement feature entirely undefined; and per BUG-001, the prototype does not even show feedback for the defined failure case (incorrect PIN), let alone these undefined ones.

Section 4: Process Improvement Suggestion (Shift-Left Testing)

What could have been done earlier

A mandatory QA-facilitated Acceptance Criteria walkthrough should have been added as an exit gate of the requirements phase, with QA manually exercising the interactive prototype screen by screen against the written acceptance criteria, before any sprint planning or development ticket was created, rather than after the spec and Figma prototype were already treated as final. A short Definition of Ready checklist, used at that walkthrough, should explicitly ask:

- Does every screen the acceptance criteria describe, including success and error states, actually exist and get triggered in the prototype?
- Does every user-facing field enforce the validation its acceptance criterion implies, with a clear error message and guidance when it does not?
- Do any two criteria describe mutually exclusive behavior for the same screen?

How it would prevent the issue

Running that checklist against this prototype would have surfaced BUG-001 (missing confirmation state) and BUG-002 (unvalidated amount field, including the over-balance case) the moment someone clicked through the flow, and BUG-003 and BUG-004 the moment someone read AC-04, AC-09, and the missing failure criteria side by side. None of that requires code, a build, or a test environment; it only requires the spec and the mockup, both of which existed before any developer picked up a ticket.

Relation to Shift-Left Testing

Shift-left testing means moving quality checks as early as possible in the delivery pipeline, ideally before development starts, instead of relying on QA to catch problems after the fact during test execution or, worse, after release. BUG-001 and BUG-002 were both found simply by clicking through the prototype by hand; BUG-003 and BUG-004 were found just by reading the spec text closely. Catching all four at the requirements stage is dramatically cheaper than catching them later, when a missing success confirmation or an unvalidated amount field could mean a real user has no idea whether they were charged, or is charged an amount they never intended.